Práctica 1: Primeros pasos con Pyomo

1. Introducción

En esta práctica se instalará todo lo necesario para poder trabajar con el paquete Pyomo de Python. Aunque existen diversos procedimientos para instalar paquetes y módulos en Python, aquí lo realizaremos utilizando las siguientes características:

- Se partirá de una instalación de Python realizada con **Anaconda** (https://www.anaconda.com/download).
- Se trabajará directamente desde el ***prompt* de Anaconda**.
- Se utilizará `conda` para instalar los paquetes y modulos necesarios.
- Para evitar problemas de privilegios de root/administrador y de interacciones no deseadas entre los paquetes y módulos que se tengan instalados se utilizará un **entorno virtual** (*environment*) para trabajar con problemas de optimización.

Independientemente de lo comentado anteriormente, se podría tener una instalación funcional de pyomo análoga a la descrita aquí utilizando otros procedimientos como `pip`, pero probablemente haya que instalar previamente alguna dependencia o programa que `conda` no requiera.

2. Instalación de Pyomo

Vamos a seguir los siguientes pasos **en el *prompt* de Anaconda** para instalar Pyomo y los optimizadores necesarios:

1. Antes de nada actualizar conda: `conda update conda -all`
2. Crear un entorno virtual vacio de nombre `OpMa` (o el que queráis): `conda create -n OpMa`
   Se puede comprobar que el entorno se ha creado correctamente: `conda env list`

```
(base) C:\Users\Alfredo>conda env list
# conda environments:
#
base                  *  C:\Users\Alfredo\anaconda3
OpMa                     C:\Users\Alfredo\anaconda3\envs\OpMa
```

3. Activar el entorno: `conda activate OpMa`
4. Actualizar conda y ya instala una serie de paquetes básicos incluido Python: `conda update conda -all`

```
(OpMa) C:\Users\Alfredo>conda update conda --all
Collecting package metadata (current_repodata.json): done
Solving environment: done

## Package Plan ##

environment location: C:\Users\Alfredo\anaconda3\envs\OpMa

added / updated specs:
- conda
```

```
The following NEW packages will be INSTALLED:

boltons            pkgs/main/win-64::boltons-23.0.0-py311haa95532_0
...
python             pkgs/main/win-64::python-3.11.5-he1021f5_0
...

Proceed ([y]/n)? y


Downloading and Extracting Packages:

Preparing transaction: done
Verifying transaction: done
Executing transaction: done
```

Se puede obtener un listado de los paquetes que están en el entorno: `conda list`

5. Instalar pyomo: `conda install -c conda-forge pyomo`
   Una vez terminado se comprueba:
   a) si la instalación ha sido correcta: `pyomo -version`

   ```
   (OpMa) C:\Users\Alfredo>pyomo --version
   Pyomo 6.6.2 (CPython 3.11.5 on Windows 10)
   ```

6. Instalar los optimizadores, aquí se usaran `glpk` e `ipopt`.
   - Instalación: `conda install -c conda-forge glpk ipopt`
   - Comprobación:

     Glpk: `glpsol -version`

     ```
     (OpMa) C:\Users\Alfredo>glpsol --version
     GLPSOL--GLPK LP/MIP Solver 5.0
     Copyright (C) 2000-2020 Free Software Foundation, Inc.

     This program has ABSOLUTELY NO WARRANTY.

     This program is free software; you may re-distribute it under the terms
     of the GNU General Public License version 3 or later.
     ```

     Ipopt: `ipopt -version`

     ```
     (OpMa) C:\Users\Alfredo>ipopt --version
     Ipopt 3.11.1 (Microsoft cl 16.00.40219.01 for x64), ASL(20130606)
     ```

     NOTA: La versión 3.14 de `ipopt` (más reciente hasta la fecha) no suele ser reconocido el ejecutable en SO Windows. También puede suceder que se obtenga algún problema en la instalación. Por ejemplo:

     ```
     "ipopt" no se reconoce como un comando interno o externo,
     programa o archivo por lotes ejecutable.
     ```

     En cualquiera de estos casos se puede instalar una versión anterior del optimizados con cualquiera de las siguientes instrucciones:
     - `conda install -c "conda-forge/label/cf202003" ipopt`
       Instala la version 3.11.1.

- `conda install -c conda-forge ipopt=3.11.1`

7. Tareas finales:
   a) Ver listado de optimizadores que podrían utilizarse: `pyomo help -s`
   b) Realizar una actualización: `conda update conda -all`
   c) Limpiar directorio del entorno: `conda clean -all -yes`
   d) Salir del entorno y volver al entorno base: `conda deactivate`

3. Uso del servidor NEOS

Si no se dispone de los optimizadores instalados en el ordenar se podrían utilizar algunos de los que ofrece el servidor NEOS (https://neos-server.org/neos/). En este caso es necesario crear una variable de entorno llamada `NEOS_EMAIL` donde se almacene una dirección de correo electrónico a donde se enviarán los resultados que se obtengan. Para ello se puede proceder de cualquiera de las siguientes formas:

- Ejecutar en el *prompt* de Anaconda: `conda env config vars set NEOS_EMAIL=tu_direccion@ema.il`
  Esto solo es necesario hacerlo una vez, después ya queda almacenada como variable de entorno.
- Añadir las siguientes lineas dentro de cada script de Python:

```
import os
os.environ['NEOS_EMAIL'] = 'tu\_direccion@ema.il'
```

- En ambos casos substituir `tu_direccion@ema.il` por una dirección de correo válida.

4. Primer ejemplo

A continuación se verá como interaccionar con Pyomo para obtener la solución a un primer problema de optimización. Para ello, Pyomo necesitará los siguientes componentes:

- Un fichero donde esté almacenado el modelo/problema concreto (con datos) o el modelo/problema abstracto (genérico)
- Un fichero donde estén almacenados los datos (solo es necesario en el caso de un modelo/problema abstracto)
- Seleccionar el optimizador adecuado para el problema a resolver

Una vez obtenido estos datos, simplemente se ejecuta en el *prompt* de Anaconda uno de los siguientes comandos dependiendo de si se dispone de un problema concreto o abstracto, respectivamente

- Modelo concreto: `pyomo solve --solver='nombre_solver' --summary mi_modelo.py`
- Modelo abstracto `pyomo solve --solver='nombre_solver' --summary mi_model.py mis_datos.dat`

El resultado de la optimización se almacena en el fichero `results.yml` y, al añadir la opción `--summary`, también lo muestra en pantalla.

4.1. Uso del servidor NEOS

Si se quiere usar el servidor NEOS, teniendo ya creada la variable de entorno `NEOS_EMAIL` habría que añadir la opción

```
--solver-manager=neos
```

a las líneas de comandos anteriores

4.2. Ejemplo Pinturas

Ejemplo 4.1. *Una empresa fabrica pinturas para interiores y exteriores. La tabla siguiente proporciona los datos básicos del problema:*

	Pintura de exteriores	***Pintura de interiores***	***Disponibilidad diaria***
Materia prima A (ton.)	*6*	*4*	*24*
Materia prima B (ton.)	*1*	*2*	*6*
Beneficio por ton. producida (miles de euros)	*5*	*4*	

Un encuesta de mercado indica que la demanda diaria de pintura para interiores no puede ser mayor que una tonelada más que la de pintura para exteriores. También que la demanda máxima de pintura para interiores es de 2 toneladas.

La empresa desea determinar ***la mezcla de producción óptima*** *de pinturas para exteriores y para interiores que maximice su beneficio total diario.*

Solución: La formulación de este problema es la siguiente:

$$
\begin{array}{rl}
\textbf{máx} & z = 5x_1 + 4x_2 \\
\textbf{sujeto a} & \\
& \begin{array}{rcrcr} 6x_1 & + & 4x_2 & \leq & 24 \\ x_1 & + & 2x_2 & \leq & 6 \\ -x_1 & + & x_2 & \leq & 1 \\ & & x_2 & \leq & 2 \end{array} \\
& x_1, x_2 \geq 0
\end{array}
$$

donde $x_1, x_2 \in \mathbb{R}$ representan, respectivamente, las toneladas fabricadas de pintura de exteriores e interiores.

Almacenamos el problema concreto en el fichero `pinturas_concrete.py` (se puede descargar de la página web de la asignatura) y su contenido será:

```
import pyomo.environ as pyo

# Crear el modelo
model = pyo.ConcreteModel()

# Definir las variables y su dominio
model.x1 = pyo.Var(domain=pyo.NonNegativeReals)
model.x2 = pyo.Var(domain=pyo.NonNegativeReals, bounds= (0, 2))

# Establecer la función objetivo
model.OBJ = pyo.Objective(expr = 5*model.x1 + 4*model.x2, sense=pyo.maximize)

# Establecer las restricciones
model.Constraint1 = pyo.Constraint(expr = 6*model.x1 + 4*model.x2 <= 24)
model.Constraint2 = pyo.Constraint(expr = model.x1 + 2*model.x2 <= 6)
model.Constraint3 = pyo.Constraint(expr = -model.x1 + model.x2 <= 1)
# model.Constraint4 = pyo.Constraint(expr =  model.x2 <= 2) # Se ha puesto como una cota (bound)
```

Resolución del problema utilizando Pyomo:

- **Optimizadores locales:** Se utilizará el optimizador `glpk` y para ello se tiene que ejecutar el siguiente código en el *prompt* de Anaconda:

  ```
  pyomo solve --solver=glpk --summary pinturas_concrete.py
  ```

  El resultado se almacenará en el fichero `result.yml` y por pantalla se mostrará:

```
[    0.00] Setting up Pyomo environment
[    0.00] Applying Pyomo preprocessing actions
[    0.00] Creating model
[    0.00] Applying solver
[    0.08] Processing results
Number of solutions: 1
Solution Information
Gap: 0.0
Status: feasible
Function Value: 21.0
Solver results file: results.yml

==========================================================
Solution Summary
==========================================================

Model unknown

Variables:
x1 : Size=1, Index=None
Key  : Lower : Value : Upper : Fixed : Stale : Domain
None :     0 :   3.0 :  None : False : False : NonNegativeReals
x2 : Size=1, Index=None
Key  : Lower : Value : Upper : Fixed : Stale : Domain
None :     0 :   1.5 :     2 : False : False : NonNegativeReals

Objectives:
OBJ : Size=1, Index=None, Active=True
Key  : Active : Value
None :   True :  21.0

Constraints:
Constraint1 : Size=1
Key  : Lower : Body : Upper
None :  None : 24.0 :  24.0
Constraint2 : Size=1
Key  : Lower : Body : Upper
None :  None :  6.0 :   6.0
Constraint3 : Size=1
Key  : Lower : Body : Upper
None :  None : -1.5 :   1.0

[    0.08] Applying Pyomo postprocessing actions
[    0.08] Pyomo Finished
```

- **Servidor NEOS:** Se utilizará, por ejemplo, el optimizador `cplex` y se tiene que ejecutar el siguiente código en el *prompt* de Anaconda[1]:

  ```
  pyomo solve --solver-manager=neos --solver=cplex --summary pinturas_concrete.py
  ```

  El resultado se enviará a la dirección de email y por pantalla se mostrará:

  ```
  [    0.00] Setting up Pyomo environment
  ```

[1] Recoradr haber generado primero la variable de entorno `NEOS_EMAIL` con una dirección de correo válida

```
[    0.00] Applying Pyomo preprocessing actions
[    0.00] Creating model
[    0.00] Applying solver
[    3.13] Processing results
Number of solutions: 1
Solution Information
Gap: None
Status: optimal
Function Value: 21.0
Solver results file: results.yml

============================================================
Solution Summary
============================================================

Model unknown

Variables:
x1 : Size=1, Index=None
Key  : Lower : Value : Upper : Fixed : Stale : Domain
None :     0 :   3.0 :  None : False : False : NonNegativeReals
x2 : Size=1, Index=None
Key  : Lower : Value : Upper : Fixed : Stale : Domain
None :     0 :   1.5 :     2 : False : False : NonNegativeReals

Objectives:
OBJ : Size=1, Index=None, Active=True
Key  : Active : Value
None :   True :  21.0

Constraints:
Constraint1 : Size=1
Key  : Lower : Body : Upper
None :  None : 24.0 :  24.0
Constraint2 : Size=1
Key  : Lower : Body : Upper
None :  None :  6.0 :   6.0
Constraint3 : Size=1
Key  : Lower : Body : Upper
None :  None : -1.5 :   1.0

[    3.15] Applying Pyomo postprocessing actions
[    3.15] Pyomo Finished
```

□

5. Ejercicios

1. Dado el siguiente problema de programación lineal:

$$\begin{array}{rl} \text{mín} & z = 2x_1 - 8x_2 \\ \textbf{sujeto a} & \\ & \begin{array}{rcrcrcl} & & 6x_1 & + & 7x_2 & \leq & 7 \\ & & 2x_1 & - & 8x_2 & \leq & 6 \\ - & & 6x_1 & + & 8x_2 & \leq & 8 \end{array} \\ & \boldsymbol{x} \geq \mathbf{0}. \end{array}$$

   a) Resolverlo a mano utilizando el método gráfico

   b) Resolverlo utilizando Pyomo.

2. Dado el siguiente problema de programación lineal: Dado el siguiente problema de programación lineal:

$$\begin{array}{rl} \text{mín} & z = -4x_1 + 7x_2 + 4x_3 + 4x_4 \\ \textbf{sujeto a} & \\ & \begin{array}{rcrcrcrcl} - & 3x_1 & + & 2x_2 & + & 2x_3 & - & 2x_4 & \geq & 1 \\ - & 3x_1 & - & 8x_2 & + & 5x_3 & - & x_4 & = & 7 \\ - & 4x_1 & + & 4x_2 & - & 3x_3 & - & x_4 & \leq & 8 \end{array} \\ & \boldsymbol{x} \geq \mathbf{0}. \end{array}$$

   Resolverlo utilizando Pyomo.

3. En una granja se usa diariamente un mínimo de 826 kg de un alimento especial que es una mezcla de maíz, trigo, guisante y soja. De cada uno de estos alimentos se sabe que:

   Maíz: contiene un 9 % de proteínas, un 9 % de fibra y un 76 % de hidratos de carbono, y su coste por kg es de 0.31 euros,

   Trigo: contiene un 32 % de proteínas, un 14 % de fibra y un 51 % de hidratos de carbono, y su coste por kg es de 0.36 euros,

   Guisante: contiene un 5 % de proteínas, un 7 % de fibra y un 14 % de hidratos de carbono, y su coste por kg es de 0.42 euros,

   Soja: contiene un 40 % de proteínas, un 23 % de fibra y un 8 % de hidratos de carbono, y su coste por kg es de 0.53 euros,

   Las necesidades dietéticas del alimento especial tienen que contener un mínimo de 10 % de proteínas, entre un 14 % y un 22 % de fibras y un máximo de 59 % de hidratos de carbono. La granja desea determinar las cantidades de la mezcla que proporcionen un coste diario mínimo.

   a) Escribe el modelo matemático que se ajusta al problema de optimización

   b) Resuelve el problema utilizando Pyomo.

4. Dado el siguiente problema de programación lineal:

$$\begin{array}{rl} \text{mín} & z = -5x_2 \\ \textbf{sujeto a} & \\ & \begin{array}{rcrcrcl} - & 4x_1 & + & 6x_2 & \leq & 8 \\ & 4x_1 & - & 8x_2 & \leq & 8 \\ & 3x_1 & - & 4x_2 & \leq & 4 \end{array} \\ & \boldsymbol{x} \geq \mathbf{0}. \end{array}$$

   a) Resolverlo a mano utilizando el método gráfico

b) Resolverlo utilizando Pyomo.

5. Dado el siguiente problema de programación lineal:

$$\begin{array}{rl} \text{mín} & z = 3x_1 - 2x_2 + 7x_3 + 8x_4 \\ \textbf{sujeto a} & \\ & \begin{array}{rcrcrcrcl} - & 2x_1 & - & 6x_2 & - & 2x_3 & & \leq & 1 \\ - & 2x_1 & + & 7x_2 & - & 6x_3 & - \; 5x_4 & \geq & 1 \\ - & x_1 & - & 4x_2 & + & 8x_3 & + \; 7x_4 & \geq & 4 \end{array} \\ & \boldsymbol{x} \geq \mathbf{0}. \end{array}$$

Resolverlo utilizando Pyomo.

6. En una granja se usa diariamente un mínimo de 924 kg de un alimento especial que es una mezcla de maíz, trigo, guisante y soja. De cada uno de estos alimentos se sabe que:

Maíz: contiene un 9 % de proteínas, un 9 % de fibra y un 72 % de hidratos de carbono, y su coste por kg es de 0.33 euros,

Trigo: contiene un 34 % de proteínas, un 13 % de fibra y un 48 % de hidratos de carbono, y su coste por kg es de 0.37 euros,

Guisante: contiene un 4 % de proteínas, un 9 % de fibra y un 13 % de hidratos de carbono, y su coste por kg es de 0.45 euros,

Soja: contiene un 46 % de proteínas, un 20 % de fibra y un 7 % de hidratos de carbono, y su coste por kg es de 0.57 euros,

Las necesidades dietéticas del alimento especial tienen que contener un mínimo de 13 % de proteínas, entre un 15 % y un 21 % de fibras y un máximo de 56 % de hidratos de carbono. La granja desea determinar las cantidades de la mezcla que proporcionen un coste diario mínimo.

a) Escribe el modelo matemático que se ajusta al problema de optimización

b) Resuelve el problema utilizando Pyomo.